COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering Process Scheduling, Memory Management & Disk Scheduling

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE AIML
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Date of Issue	21/08/2026
Date of Submission	2/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Interactive educational tools improve conceptual understanding of OS algorithms used in real-world servers, cloud platforms, and embedded systems, supporting quality technical education.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an interactive Operating Systems simulator that applies core CSA04 concepts — process scheduling (Units I–II), memory management with paging and page replacement (Unit III), and disk scheduling (Unit IV) — to a unified educational training platform called **OSLearn**. Students must integrate multiple algorithm families, produce clear comparative visualizations and step-by-step traces, deploy the system as a lightweight web application using Streamlit, validate correctness and usability with sample test cases, and address basic responsible-computing considerations such as transparency of intermediate steps and reproducibility of results.

C. Problem Statement

Scenario: "OSLearn" – Interactive OS Algorithm Trainer

Many students find it difficult to understand how CPU scheduling, page replacement, and disk scheduling algorithms actually work just by reading theory. Your task is to design and implement an interactive educational web application called **OSLearn** that helps students practice, visualize, and compare these core Operating Systems algorithms through a unified interface.

The system shall integrate three major OS algorithm families:

1. Process Scheduling

Implement FCFS, SJF (Non-preemptive), and Round Robin.

- Users should be able to enter process details (Process ID, Arrival Time, Burst Time, and Time Quantum for Round Robin).

- The system must display a Gantt chart / timeline visualization together with a table showing Waiting Time, Turnaround Time, and Average Waiting/Turnaround Time for each algorithm, enabling side-by-side comparison.

2. Memory Management (Page Replacement)

Implement FIFO and LRU page-replacement algorithms.

- Users will enter the number of frames and a page reference string.
- The system should show the step-by-step frame status after each reference, highlight page faults, and report the total number of page faults for each algorithm so that students can compare performance.

3. Disk Scheduling

Implement FCFS, SSTF, and SCAN.

- Users will enter the disk request queue and the initial head position.
- The system should display the order of servicing (seek sequence) and the total seek time for each algorithm, allowing clear comparative analysis.

Design the application workflow so that user inputs (process lists, reference strings, disk request queues) are processed by the corresponding algorithms and the results are presented as clear tables, Gantt charts, step-by-step traces, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate correctness. Optionally, students may add brief natural-language explanations of algorithm steps (no external LLM required).

Evaluate the application under suitable test cases and analyze correctness of algorithm results, usability of the interface, and clarity of visualizations. The completed system should also briefly address responsible-computing considerations including transparency of intermediate simulation steps, reproducibility of results, and educational ethics in presenting algorithm behavior.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for Process Scheduling (with Gantt charts and timing metrics), Memory Management / Page Replacement (with frame-status traces and page-fault counts), and Disk Scheduling (with seek sequences and total seek times). Accept user inputs as well as sample data. Produce comparative tables, charts, and step-by-step traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** Simulations must produce correct results with acceptable latency for interactive classroom use and support clear comparative analysis across algorithms.
- **Technical constraint:** Use Python + Streamlit. No database, no complex architecture, and no mandatory external LLM integration. Open-source tools only.
- **Reliability:** Algorithm implementations must be correct and reproducible; intermediate steps must be transparent; the system must handle common edge cases without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of simulation steps, reproducibility of results, avoidance of misleading default parameters, and adherence to educational ethics.
- **Evaluation & Submission:** Test all three modules with sample inputs and show correct results. Submit working Streamlit code, a short report containing sample outputs / screenshots, comparative analysis, and a GitHub repository link.

1. Problem Understanding and Formulation

Problem Understanding:

The given problem focuses on designing an integrated Operating Systems solution for CampusConnect, a university platform where many students and faculty members simultaneously access shared academic files such as gradebooks, assignments, lecture slides, and videos. The system must efficiently manage concurrent file access, physical memory, and disk operations while avoiding race conditions, deadlocks, excessive page faults, and long disk waiting times.

Specific Problems Identified

1. Process Synchronization
    - Multiple users may read the same file at the same time.
    - Faculty members need to update shared files safely.
    - The system must prevent race conditions and deadlocks.
    - Writers should not be continuously delayed by incoming readers.
2. Memory Management
    - Up to 300 user sessions may run concurrently.
    - Physical memory is limited, so paging is required.
    - Poor page replacement or excessive frame allocation can lead to thrashing and high page-fault rates.
3. File System and Disk Scheduling
    - The platform handles both very small files and multi-gigabyte lecture videos.
    - The file-allocation method should avoid external fragmentation and provide good random access.

- Disk scheduling should reduce total seek time while providing fair access to requests.

Problem Formulation

The problem can therefore be formulated as:

Design an integrated Operating Systems resource-management solution for CampusConnect that provides safe and fair synchronization of shared files, efficient memory utilization for up to 300 concurrent sessions, and fast disk access for mixed-size file workloads, while minimizing page faults, avoiding deadlocks and race conditions, and reducing disk seek time.

The major assumptions include 8 GB physical memory, 4 KB page size, 64 MB logical address space per representative session, 200 disk cylinders, and 300 concurrent sessions.

Expected outcomes:

- Deadlock-free and race-free file access.
- Low page-fault rates and reduced risk of thrashing.
- Efficient file storage and fast disk access.
- Fair and reliable service for students and faculty

2. Application of Course Knowledge (CO2, CO3, CO4)

Part I – Process Synchronization & Deadlock (CO2, Unit II)

2.1 Readers–Writers model with semaphores

Concurrent access to a shared academic file (e.g., a gradebook read by many students, written occasionally by a faculty member) is modelled as the classical Readers–Writers problem. A plain 'readers-preference' solution risks starving writers indefinitely while readers keep arriving, which is unacceptable for a system that must remain reliable. The pseudocode below adds a turnstile semaphore so that arrivals — readers and writers alike — are served in first-come-first-served order, guaranteeing mutual exclusion for writers and preventing writer starvation.

```
semaphore mutex      = 1     // protects read_count
semaphore rw_mutex    = 1     // exclusive lock on the shared file
semaphore turnstile   = 1     // FIFO fairness gate
int read_count = 0

Reader():
```

```
    wait(turnstile); signal(turnstile)   // join the FIFO queue, then pass through
    wait(mutex)
        read_count++
        if read_count == 1: wait(rw_mutex)   // first reader locks out writers
    signal(mutex)
    // ---- critical section: read shared file ----
    wait(mutex)
        read_count--
        if read_count == 0: signal(rw_mutex) // last reader releases the lock
    signal(mutex)

Writer():
    wait(turnstile)
        wait(rw_mutex)                       // exclusive access
    signal(turnstile)
    // ---- critical section: write shared file ----
    signal(rw_mutex)
```

Mutual exclusion holds because rw_mutex is held by either one writer or the group of active readers, never both; race-free counting of read_count is guaranteed by mutex; and the turnstile ensures no process waits forever behind an unbounded stream of same-type requests.

2.2 Banker's Algorithm — safety check for P1–P4

Four resource types are contended: R1 = database handles (6 total), R2 = print-spooler slots (5), R3 = network sockets (7), R4 = file locks (6). A representative Allocation and Max-claim matrix is constructed for P1–P4:

Process	Allocation (R1,R2,R3,R4)	Max (R1,R2,R3,R4)	Need = Max − Allocation
P1	1, 2, 2, 1	3, 3, 2, 2	2, 1, 0, 1
P2	2, 0, 0, 1	3, 1, 1, 2	1, 1, 1, 1
P3	1, 1, 1, 0	2, 2, 3, 1	1, 1, 2, 1
P4	1, 1, 3, 2	2, 2, 4, 3	1, 1, 1, 1

Total resources = (6, 5, 7, 6). Total allocated = (5, 4, 6, 4). Available = Total − Allocated = (1, 1, 1, 2).

Safety Algorithm trace

Step	Available before	Process checked	Need ≤ Available?	Available after finish
1	1, 1, 1, 2	P2 need (1,1,1,1)	Yes → P2 runs & releases (2,0,0,1)	3, 1, 1, 3
2	3, 1, 1, 3	P1 need (2,1,0,1)	Yes → P1 runs & releases (1,2,2,1)	4, 3, 3, 4
3	4, 3, 3, 4	P3 need (1,1,2,1)	Yes → P3 runs & releases (1,1,1,0)	5, 4, 4, 4
4	5, 4, 4, 4	P4 need (1,1,1,1)	Yes → P4 runs & releases (1,1,3,2)	6, 5, 7, 6

Result: the system is in a SAFE state, with safe sequence P2 → P1 → P3 → P4.

2.3 Choice of deadlock-handling strategy

Deadlock Avoidance via the Banker's Algorithm is recommended for CampusConnect. Resource requests per session are small and largely predictable in advance (a session needs at most one database handle, one spooler slot, one socket, one file lock), so declaring a maximum claim is realistic — this is exactly the assumption Avoidance requires. Prevention (e.g., enforcing a strict global lock-acquisition order across four

independently-used resource types) would be brittle to enforce across CampusConnect's separately developed modules. Detection & Recovery would allow a deadlock to actually occur and then roll back a session, risking loss of a student's in-progress work — unacceptable given the reliability requirement in Section D. Avoidance keeps the number of resource types small (four), so the safety check's overhead is negligible relative to the reliability it buys.

Part II – Memory Management (CO3, Unit III)

2.4 Frames and page-table sizing

- Physical memory = 8 GB = 2^33 bytes; page size = 4 KB = 2^12 bytes ⇒ number of frames = 2^33 / 2^12 = 2^21 = 2,097,152 frames.
- Representative process, logical address space = 64 MB = 2^26 bytes ⇒ number of pages = 2^26 / 2^12 = 2^14 = 16,384 pages.
- Each virtual address therefore needs 14 bits of page number + 12 bits of offset (26 bits total = 64 MB); each physical address needs 21 bits of frame number + 12 bits of offset (33 bits total = 8 GB) — consistent with the values above.

A page-table entry (PTE) needs the 21-bit frame number plus valid/dirty/referenced/protection bits, which rounds to a 4-byte entry. A single flat table would need 16,384 × 4 B = 64 KB per process — workable for one process, but wasteful across 300 concurrent sessions with mostly-sparse address spaces (large unused gaps between heap and stack). The recommended design is therefore a two-level hierarchical page table: the 14-bit page number is split into a 7-bit outer index (128 entries) and a 7-bit inner index (128 entries per second-level table); inner tables are allocated only for regions the process actually uses, so idle/unused regions of the address space cost nothing in page-table memory — a good fit for many lightweight concurrent sessions.

2.5 Page-replacement comparison (FIFO vs LRU vs Optimal)

Reference string (12 references, representative of a user session accessing pages while editing and switching between a few open resources): 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, with 3 frames available.

Algorithm	Total page faults (of 12 refs)	Hits
FIFO	9	3
LRU	10	2
Optimal	7	5

Optimal gives the theoretical lower bound (7 faults), as expected, because it evicts the page used farthest in the future. Interestingly, plain FIFO (9 faults) slightly outperforms plain LRU (10 faults) for this specific reference string — a reminder that no non-lookahead policy dominates for every access pattern; LRU's advantage over FIFO is an average-case property that depends on a workload exhibiting temporal locality, not a guarantee for any single string.

2.6 Demand paging, working sets, and thrashing at 300 sessions

Demand paging loads a page only on first reference, avoiding wasted I/O for the large parts of a session's address space (unused library code, unopened files) that are never touched — important because CampusConnect sessions vary hugely in behaviour, from a student briefly viewing a grade to one uploading a large video. The working-set model tracks, for each process, the set of pages referenced within a recent window Δ and keeps exactly that set resident; if the sum of all sessions' working sets exceeds available frames, the system is at risk of thrashing (high fault rate, falling CPU utilization) and should apply load control — suspending or deferring the lowest-priority or longest-idle sessions until the remaining working sets fit in memory, rather than over-committing frames to every session. For CampusConnect's 300-session peak, a window Δ covering roughly the last 50–100 references (a few seconds of activity) is recommended, with a

baseline allocation of 8–16 frames for typical light sessions (viewing/editing text) and up to 32 frames for heavier sessions (video upload/preview), reallocated dynamically as each session's measured working-set size changes.

Part III – File Systems & Disk Scheduling (CO4, Unit IV)

2.7 File-allocation method comparison

Method	Strength for CampusConnect	Weakness for CampusConnect
Contiguous	Fast sequential/random access — good for streaming large lecture videos	External fragmentation; hard to grow a file in place (gradebooks grow over a term)
Linked	No external fragmentation; easy to grow small text files	Poor random access (bad for video scrubbing); a single broken pointer can lose the rest of the file
Indexed (inode-style)	Good random access and dynamic growth for both tiny and huge files via direct + indirect index blocks; no external fragmentation	Small extra storage/lookup overhead for index blocks

Recommendation: Indexed allocation, Unix-inode style, is the best fit. CampusConnect's workload spans a few-KB text submission (served well by a handful of direct blocks) to multi-gigabyte lecture videos (served by single/double/triple indirect blocks) — a pure contiguous scheme wastes space and fights fragmentation as gradebook-style files grow, and a pure linked scheme is unusable for video scrubbing, which needs fast random access.

2.8 Disk-scheduling comparison

Disk modelled with cylinders 0–199, read/write head starting at cylinder 50, pending request queue (8 requests): 82, 170, 43, 140, 24, 16, 190, 150.

Algorithm	Service sequence	Total head movement (cylinders)
FCFS	50→82→170→43→140→24→16→190→150	682
SCAN	50→82→140→150→170→190→199→43→24→16	332
C-SCAN	50→82→140→150→170→190→199→0→16→24→43	391

SCAN gives the lowest total seek time (332), roughly 51% less than FCFS (682) and about 15% less than C-SCAN (391), because it services requests in cylinder order in one sweep instead of jumping back and forth as they were submitted.

2.9 Relation to Unix file-system concepts

The recommended indexed allocation mirrors the Unix inode: an inode stores a handful of direct block pointers (covering most small text/gradebook files without any indirection) plus single-, double-, and triple-indirect pointers that scale up to multi-gigabyte video files without a single oversized index block. The Unix buffer/page cache complements this: frequently accessed inodes and data blocks — e.g., the current week's assignment files — stay cached in memory, which both cuts the number of physical disk requests the scheduler ever sees and lets writes (e.g., gradebook updates) be batched, reducing the total head movement the SCAN policy above has to service.

3. Solution / Design / Methodology

The proposed CampusConnect OS resource-management design integrates synchronization, memory management, and file-system/disk scheduling into one system. The design focuses on providing safe concurrent access, efficient memory utilization, and fast disk access. The assignment uses Python-based algorithm implementations, with Streamlit as an optional interface for interactive simulation.

3.1 Synchronization Design

1. A fair Readers–Writers algorithm using semaphores is used to protect shared academic files such as gradebooks and assignment files.
2. The rw_mutex ensures that writers get exclusive access while multiple readers can read simultaneously.
3. A turnstile semaphore is used to maintain FIFO fairness and prevent writer starvation.
4. The Banker's Algorithm is used for deadlock avoidance among shared resources such as database handles, print-spooler slots, network sockets, and file locks.

3.2 Memory Management Design

1. The system uses two-level hierarchical paging for a 64 MB logical address space.
2. With 8 GB physical memory and a 4 KB page size, the system has 2,097,152 physical frames.
3. The 14-bit page number is divided into a 7-bit outer index and 7-bit inner index.
4. Clock (Second-Chance) page replacement is selected as the practical replacement policy because it provides an efficient approximation of LRU.
5. A working-set-based frame allocation method is used to reduce the possibility of thrashing when up to 300 sessions are active.

3.3 File-System and Disk-Scheduling Design

1. Unix inode-style indexed allocation is selected for storing both small academic files and large lecture videos.
2. Indexed allocation provides good random access and supports dynamic file growth without external fragmentation.
3. SCAN disk scheduling is selected as the default disk-scheduling policy.
4. A buffer/page cache is used to keep frequently accessed files and blocks in memory, reducing unnecessary disk operations.

3.4 Implementation Methodology

The algorithms are implemented and tested using Python. The main implementations include:

- Banker's Algorithm – checks whether the system is in a safe state.
- FIFO, LRU and Optimal – simulate page replacement and calculate page faults.

- FCFS, SCAN and C-SCAN – simulate disk scheduling and calculate total head movement.
- POSIX threads and semaphores – demonstrate the Readers–Writers synchronization problem on Linux/WSL.

This methodology allows each OS concept to be implemented, tested, compared, and demonstrated through intermediate steps and measurable results.

3.5 Architecture / flow diagram

The diagram below shows how the three subsystems interact when a user session accesses a shared file: a request first passes through the synchronization layer (which also runs the Banker's safety check), then through the memory manager (which pages in the session's data), and finally reaches the file system and disk scheduler that actually service the shared file on disk.

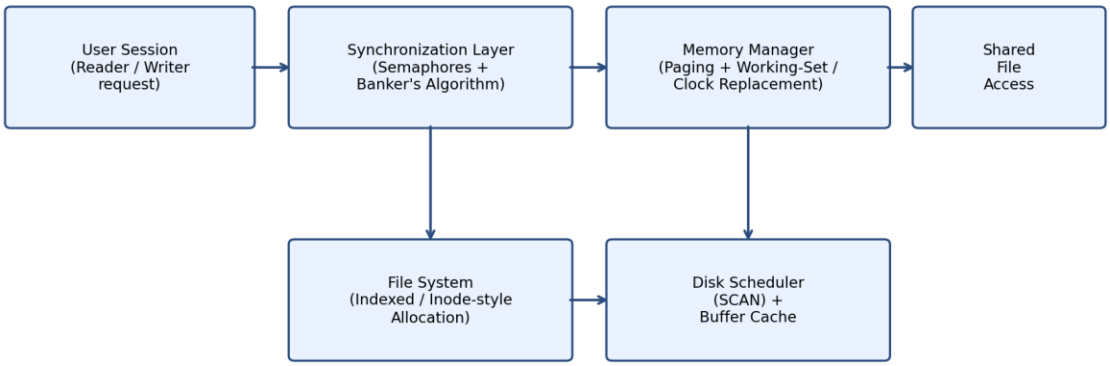

Figure 1: Simplified OS resource-management flow for a shared-file session in CampusConnect

4. Use of Modern Tools

The three algorithm families were implemented and cross-checked in Python (the same logic that would back the OSLearn Streamlit modules). Representative source code is given below; running each script reproduces the tables in Sections 2 and 5.

4.1 Banker's Algorithm (safety check)

```
def is_safe(available, allocation, need):
    n = len(allocation)          # number of processes
    m = len(available)           # number of resource types
    work = list(available)
    finish = [False] * n
    sequence = []
    while len(sequence) < n:
        progressed = False
        for p in range(n):
            if not finish[p] and all(need[p][j] <= work[j] for j in range(m)):
```

```
                work = [work[j] + allocation[p][j] for j in range(m)]
                finish[p] = True
                sequence.append(p)
                progressed = True
        if not progressed:
            return False, sequence   # unsafe -- no process could proceed
    return True, sequence
```

4.2 Page-replacement simulator (FIFO / LRU / Optimal)

```
def fifo(refs, nframes):
    frames, faults = [], 0
    for r in refs:
        if r not in frames:
            faults += 1
            if len(frames) >= nframes: frames.pop(0)
            frames.append(r)
    return faults

def lru(refs, nframes):
    frames, faults = [], 0
    for r in refs:
        if r in frames:
            frames.remove(r); frames.append(r)
        else:
            faults += 1
            if len(frames) >= nframes: frames.pop(0)
            frames.append(r)
    return faults

def optimal(refs, nframes):
    frames, faults = [], 0
    for i, r in enumerate(refs):
        if r in frames: continue
        faults += 1
        if len(frames) < nframes:
            frames.append(r)
        else:
            future = refs[i+1:]
            evict = max(frames, key=lambda f: future.index(f) if f in future else 1e9)
            frames.remove(evict); frames.append(r)
    return faults
```

4.3 Disk-scheduling simulator (FCFS / SCAN / C-SCAN)

```
def fcfs(requests, head):
    seq = [head] + requests
    return sum(abs(seq[i+1]-seq[i]) for i in range(len(seq)-1)), seq

def scan(requests, head, disk_max=199):
    right = sorted(r for r in requests if r >= head)
    left  = sorted((r for r in requests if r < head), reverse=True)
    seq = [head] + right + [disk_max] + left
    return sum(abs(seq[i+1]-seq[i]) for i in range(len(seq)-1)), seq

def cscan(requests, head, disk_max=199):
    right = sorted(r for r in requests if r >= head)
    left  = sorted(r for r in requests if r < head)
    seq = [head] + right + [disk_max, 0] + left
```

```
    return sum(abs(seq[i+1]-seq[i]) for i in range(len(seq)-1)), seq
```

4.4 pthread demonstration (Linux / WSL)

On a Linux VM or WSL, the same Readers–Writers pseudocode (Section 2.1) can be demonstrated with POSIX semaphores and threads:

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

sem_t mutex, rw_mutex, turnstile;
int read_count = 0;

void *reader(void *arg) {
    sem_wait(&turnstile); sem_post(&turnstile);
    sem_wait(&mutex);
    if (++read_count == 1) sem_wait(&rw_mutex);
    sem_post(&mutex);
    printf("Reader %ld is reading\n", (long)arg);
    sem_wait(&mutex);
    if (--read_count == 0) sem_post(&rw_mutex);
    sem_post(&mutex);
    return NULL;
}

void *writer(void *arg) {
    sem_wait(&turnstile);
    sem_wait(&rw_mutex);
    sem_post(&turnstile);
    printf("Writer %ld is writing\n", (long)arg);
    sem_post(&rw_mutex);
    return NULL;
}
```

(Compile with gcc -pthread; run and capture terminal output as the evidence screenshot for submission, alongside the Python script outputs.)

5. Results and Validation

5.1 Page-fault comparison (3 frames, 12-reference string)

Algorithm	Page faults
FIFO	9
LRU	10
Optimal	7

5.2 Total seek time comparison (200 cylinders, head = 50, 8 requests)

Algorithm	Total head movement
FCFS	682
SCAN	332
C-SCAN	391

5.3 Banker's Algorithm safety trace

Reproduced from Section 2.2: Available = (1,1,1,2) → safe sequence found: P2 → P1 → P3 → P4. The system is confirmed to be in a safe state, so no deadlock can occur under the stated maximum claims.

5.4 Validation of the synchronization design

- Mutual exclusion: rw_mutex is acquired either by the writer or (indirectly) by the first reader, so a writer can never run concurrently with an active reader group — no race condition on the shared file.
- No starvation: the turnstile semaphore forces FIFO ordering of arrivals, so a writer cannot be blocked forever by a continuous stream of new readers.
- No deadlock under the stated resource matrix: the Banker's Algorithm found an explicit safe sequence (Section 5.3), so, given the declared maximum claims, the four processes can always be driven to completion without circular wait.

6. Analysis and Engineering Decision

6.1 Page-replacement recommendation

Optimal is the theoretical best (7 faults) but is not implementable, since it requires future knowledge of references. Plain LRU (10 faults) even slightly underperformed FIFO (9 faults) on the constructed reference string, showing that LRU's usual edge over FIFO is a locality-dependent, average-case property rather than a per-string guarantee. Given this, and that true LRU needs per-access bookkeeping (a timestamp or move-to-front update on every reference — expensive at OS scale), the practical recommendation for CampusConnect is the Clock (second-chance) algorithm: it approximates LRU's recency information with an O(1) reference-bit check per replacement, giving near-LRU fault behaviour without LRU's overhead — the right trade-off for a system serving hundreds of concurrent sessions.

6.2 Disk-scheduling recommendation

SCAN produced the lowest total seek time (332), about 51% less than FCFS (682) and about 15% less than C-SCAN (391). The trade-off is fairness: SCAN services cylinders near the current sweep more often than those just passed, so requests near the 'far' end of a long sweep can wait longer, whereas C-SCAN always sweeps in the same direction and gives more uniform response times at a somewhat higher total-movement cost. For CampusConnect, where raw throughput matters most under bursty peak load (e.g., many simultaneous submissions near a deadline) and the request queue is refreshed frequently enough that no cylinder is starved for long, SCAN is recommended as the default policy.

6.3 Deadlock-strategy trade-off

Avoidance (chosen) requires processes to declare maximum claims in advance and pays a periodic safety-check cost, but this cost is small given only four resource types and modest per-session claims. Prevention would need a strict, globally agreed resource-acquisition order across independently developed CampusConnect modules — difficult to enforce and easy to violate as the system evolves. Detection & Recovery is cheaper when deadlocks are rare, but recovery typically means aborting or rolling back a process,

6.4 Justification of the integrated design

Together, the quantitative results support the stated requirements in Section D: lower page faults (Clock ≈ near-Optimal territory) and lower seek time (SCAN, −51% vs. FCFS) directly reduce average session latency, satisfying the 'acceptable latency for interactive classroom use' requirement, while the fair, deadlock-avoiding synchronization design satisfies the 'handle common edge cases without crashing' and reproducibility requirements.

7. Broader Considerations

Reliability & Safety

Mutual exclusion on writes (via rw_mutex) combined with the Banker's Algorithm's safe-state guarantee prevents both corrupted concurrent writes and deadlock-induced hangs, protecting the integrity of shared academic files such as gradebooks.

Sustainability / Economics

Working-set-based frame allocation and low-overhead Clock replacement reduce unnecessary page faults, and SCAN scheduling reduces total disk-head movement; both cut wasted CPU/disk I/O cycles, lowering the university server's energy consumption, hardware wear, and operating cost.

Accessibility & Society

The turnstile's FIFO fairness ensures no student or faculty member is indefinitely blocked from writing, and working-set-based load control lets the system degrade gracefully (admitting fewer new sessions) rather than freezing under peak load, so responsiveness stays more equitable across all concurrent users.

Ethics / Professional Responsibility

Because gradebooks and submissions are sensitive student data, the locking/synchronization layer should be paired with proper authentication and authorization above the OS layer, audit logging of writes for accountability, and a design that never silently drops or overwrites a student's work during concurrent access.

8. Conclusion

The CampusConnect project provided a practical understanding of how different Operating System concepts work together in a real-world system. The proposed design combines a fair Readers–Writers synchronization mechanism, Banker's Algorithm for deadlock avoidance, hierarchical paging with working-set management, Unix inode-style indexed file allocation, and SCAN disk scheduling.

The analysis showed that SCAN provided lower disk head movement compared with FCFS, while the page-replacement comparison demonstrated the differences between FIFO, LRU, and Optimal algorithms. The synchronization design also helps provide safe and fair access to shared academic files while avoiding race conditions and deadlock under the given assumptions.

Overall, this project helped demonstrate how Operating System algorithms can be applied to improve resource utilization, reliability, fairness, and system performance. The design provides a suitable foundation for managing concurrent users and mixed workloads in a university platform such as CampusConnect.

9. Student Reflection

9.1 What did integrating these three areas reveal beyond what was covered separately in class?

While studying Operating Systems, I initially understood synchronization, memory management, and file systems as separate topics. However, while working on this project, I understood that these concepts are closely connected in a real system. For example, if a process holds a file lock for a long time, other processes may have to wait. This can affect memory usage and also increase the number of pending disk requests.

The project helped me understand that OS algorithms should not be considered independently. A change in one resource-management area can affect the performance of other areas. Integrating all three topics gave me a better understanding of how an operating system manages resources and maintains system performance, fairness, and reliability.

9.2 What would be improved with additional time or resources?

If more time were available, I would test the system with larger and more realistic workloads instead of using only the sample reference strings and disk request queues. Real workload data could provide a better comparison of different page-replacement and disk-scheduling algorithms.

I would also improve the interactive interface by adding more visualizations and allowing users to compare algorithms more easily. Another possible improvement would be to test the file-storage design using a distributed file system with multiple servers. This would help understand how the synchronization and resource-management techniques would work in a larger real-world environment.

Overall, this project improved my understanding of Operating Systems by showing me how theoretical concepts can be combined to solve practical resource-management problems.

10. References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] CSA04 – Operating Systems, Course Lecture Notes, Units I–IV, Dept. of Computer Science and Engineering, 2026–2027.

[3] Python Software Foundation, "Python 3 documentation," [Online]. Available: https://docs.python.org/3/

[4] Streamlit Inc., "Streamlit documentation," [Online]. Available: https://docs.streamlit.io/

[5] Anthropic, "Claude," AI-assisted drafting and calculation-verification tool used in preparing this report, 2026. [Online]. Available: https://www.anthropic.com

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Scheduling, Paging, Disk)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Streamlit, Python)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	**100**

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Process Scheduling	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – Page Replacement	CO3	PO1, PO2, PO3	L3/L4	6
Application – Disk Scheduling	CO4	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated OSLearn	CO1–CO4	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Streamlit / Python)	CO1–CO4	PO5	L3	10
Validation of results	CO1–CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO4	PO6, PO12	L3/L4	10

G. CO–PO–Assessment Mapping

note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	< 50%

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____